

COMPUTING INFRASTRUCTURES

From Interaction to Deployment: Working with Language Models

2025–2026 Project

Group: 11

Members: Oliwia Wójcicka
Karolina Kawulska
Daria Bartkowiak

June 14, 2026

Contents

1	Introduction	2
1.1	Hardware Environment	2
2	Phase 1 – Local Setup & Basic Inference with Ollama	2
2.1	Setup: Installing and Running Ollama	2
2.2	Task 1: Hardware Inspection with llmfit	3
2.3	Task 2: Pull and Run Our First Model	5
2.4	Task 3: Pull and Compare a Second Model	6
2.5	Task 4: Call Ollama Directly with curl	9
2.6	Task 5: Streaming vs Non-Streaming Responses	11
2.7	Task 6: Experiment with Payload Parameters	12
2.8	Task 8: Summary of Phase 1 Findings	14
3	Optional Tasks	15
3.1	Optional Token 1: Try a Different Model Family	15
3.2	Optional Token 2: Python Script Using requests	16
4	Phase 2 – Continue.dev as a Local Coding Assistant	17
4.1	Task 1: Installing and Configuring Continue.dev	17
4.2	Task 2: Testing Chat, Edit, and Autocomplete Modes	18
4.3	Task 3: Observing Ollama Logs	21
4.4	Task 4: Assigning Different Models to Different Roles	22
5	Optional Tasks – Phase 2	23
5.1	Optional Token 1: Custom System Prompt	23
5.2	Optional Token 2: MCP Server (2 tokens)	27
6	Phase 3 – Containerising Ollama with Docker	30
6.1	Task 1: Writing the docker-compose.yml	30
6.2	Task 2: Pulling a Model Inside the Container	30
6.3	Task 3: Inspecting Container Hardware Logs	31
7	Phase 4 – Cloud Deployment on AWS EC2	31
7.1	Steps 1–4: Creating the EC2 Instance	31
7.2	Task 1: Hardware Inspection	33
7.3	Task 2: Installing Docker and Docker Compose	33
7.4	Task 3: Uploading Files and Running Containers	33
7.5	Task 5: Pulling a Small Model	34
7.6	Task 6: Adding Swap Space	34
7.7	Optional Token 6: Latency Benchmark – Local vs Cloud	34
8	Problems Encountered and Solutions	35
9	Conclusions	36
10	AI Models Used	37
11	Bibliography	37

1 Introduction

The goal of this project is to go beyond simply *using* language models and to understand the computing infrastructure that makes them work. Instead of treating an AI assistant as a magic chat box, we build the system ourselves, piece by piece: a model running locally, an API serving it, a coding assistant talking to that API, the whole thing packaged into Docker containers, and finally the same system deployed on a cloud machine.

The project is divided into four phases. In Phase 1 we run language models locally with Ollama and learn how our own hardware limits what we can run. In Phase 2 we turn that local model into a coding assistant using Continue.dev. In Phase 3 we containerise the setup with Docker, and in Phase 4 we deploy it to an AWS cloud instance. Throughout, we compare local and cloud execution and reflect on the practical tradeoffs of running an AI-powered system.

1.1 Hardware Environment

We carried out this project on a laptop with the following specifications:

Component	Value
CPU	AMD Ryzen 7 8840HS w/ Radeon 780M Graphics
CPU cores	16 logical threads (8 physical cores)
Total RAM	15.28 GB ($\approx$ 16 GB)
GPU	AMD Radeon 780M (integrated iGPU)
VRAM	0.5 GB (Vulkan backend)
Operating system	Windows 11

Table 1: Hardware used for the project.

2 Phase 1 – Local Setup & Basic Inference with Ollama

In this first phase we set up a language model running locally on our own machine. Before downloading anything, we inspected our hardware to understand what we could realistically run, and only then chose a model. The main idea of this phase is that AI applications are not just “the model”: they also depend on the hardware and the infrastructure that serves the model.

2.1 Setup: Installing and Running Ollama

We installed Ollama from <https://ollama.com/download> and ran the installation command from PowerShell, as recommended. We verified the installation with:

```
ollama --version
# ollama version is 0.30.6
```

On Windows, Ollama starts automatically in the background as a service after installation. Because of this, running `ollama serve` manually returned an error saying the port was already in use:

```
ollama serve
# Error: listen tcp 127.0.0.1:11434: bind: only one usage of each
# socket address (protocol/network address/port) is normally allowed
```

This is expected behaviour: it simply means the Ollama server is already running and listening on its default port, 11434. We confirmed the server was reachable with a quick API call:

```
curl.exe http://localhost:11434/api/version
# {"version": "0.30.6"}
```

Here localhost means our own machine and 11434 is Ollama’s default port. This same address will be used later by Continue.dev, the Docker containers, and the cloud deployment.

2.2 Task 1: Hardware Inspection with llmfit

To decide which models our machine could realistically run, we used llmfit, a terminal tool that detects the system’s CPU, RAM and GPU and scores each model by how well it fits the available hardware. On Windows we installed it through the Scoop package manager:

```
scoop install llmfit
llmfit
```

The tool automatically detected our running local Ollama server (localhost:11434), shown as Ollama: ✓ (0 installed) in the top status bar – “0 installed” because we had not pulled any model yet. Figure 1 shows the interactive TUI: the detected hardware appears in the top bar, and the models are listed below, ranked by a composite fit score.

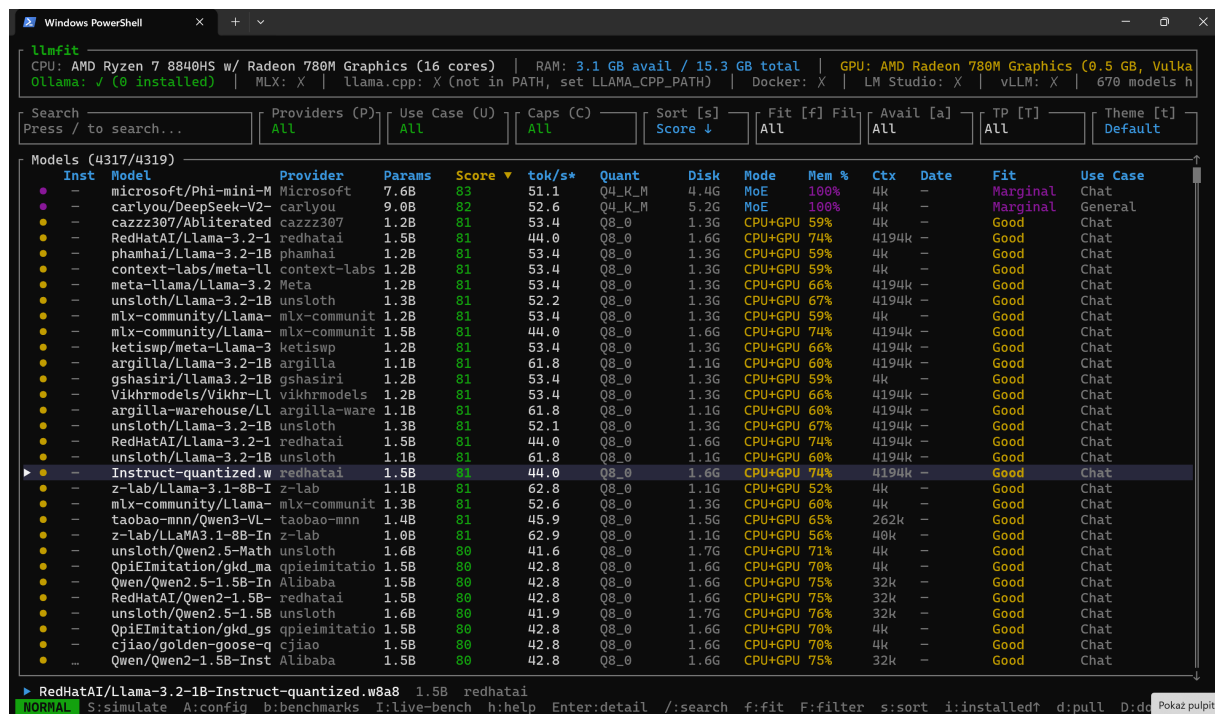


Figure 1: llmfit TUI showing the detected hardware (top bar) and models ranked by fit. No model reaches a “Perfect” rating: with only 0.5 GB of VRAM, the best small models are rated “Good” and run in CPU+GPU mode, while larger MoE models such as Phi-mini and DeepSeek-V2 are only “Marginal”.

To get clean, copy-pasteable hardware data we also ran the CLI commands llmfit system and llmfit fit -perfect:

```
=== System Specifications ===
CPU: AMD Ryzen 7 8840HS w/ Radeon 780M Graphics (16 cores)
Total RAM: 15.28 GB
Available RAM: 3.69 GB
Backend: Vulkan
```

```
GPU: AMD Radeon 780M Graphics (0.50 GB VRAM, Vulkan)
```

```
=== llmfit fit --perfect ===  
(670 models hidden -- incompatible backend)  
No compatible models found for your system.
```

Model fit results

- **Perfect fit:** none. Running `llmfit fit --perfect` returned “*No compatible models found for your system*”. A “Perfect” rating requires the model to fit entirely in VRAM on the GPU, and with only 0.5 GB of VRAM no model qualifies. On top of that, `llmfit` hid 670 models as “incompatible backend” because our GPU uses the Vulkan backend rather than CUDA.
- **Good fit:** in the TUI, small 1–1.5B parameter models (for example Llama-3.2-1B, Qwen2.5-1.5B and similar variants, Q8_0 quantization, around 1.1–1.7 GB on disk) are the best achievable rating. They all run in CPU+GPU mode rather than purely on the GPU.
- **Too large:** mixture-of-experts and larger models such as `microsoft/Phi-mini` (7.6B) and `DeepSeek-V2` (9.0B) reach only a “Marginal” rating at 100% memory usage, which makes them impractical on this machine.

Answers to the guiding questions

- **What hardware does your machine have?** An AMD Ryzen 7 8840HS CPU with 16 logical threads (8 physical cores), 15.28 GB of RAM, and an integrated AMD Radeon 780M GPU with 0.5 GB of VRAM, running on Windows.
- **Does your computer have a dedicated GPU?** No. The Radeon 780M is an integrated GPU (iGPU) that shares memory with the system, which is why only 0.5 GB of VRAM is reported.
- **How much RAM and/or VRAM is available?** 15.28 GB of total RAM (about 3.7 GB free at the time of measurement) and only 0.5 GB of VRAM.
- **Which models does llmfit recommend?** Only small 1–1.5B parameter models, rated “Good” (e.g. Llama-3.2-1B and Qwen2.5-1.5B variants). No model reaches a “Perfect” rating.
- **Why do you think those models are recommended?** Because they are the only ones small enough to run acceptably given our limited VRAM. Larger models would not fit and would either fail or run extremely slowly, so `llmfit` ranks the small ones highest.
- **What seems to be the main limitation of your machine: CPU, RAM, GPU, or VRAM?** VRAM. The CPU (16 threads) and RAM (15.28 GB) are comfortable, but the integrated GPU only exposes 0.5 GB of VRAM, which is the binding constraint and forces models into CPU+GPU mode.

Discussion

The binding limitation of our machine is the lack of a dedicated GPU: the integrated Radeon 780M exposes only 0.5 GB of VRAM over the Vulkan backend, so no model can be loaded fully on the GPU. As a result `llmfit` reports no “Perfect” fit at all, and even the best small models fall back to CPU+GPU execution. Our RAM (15.28 GB) and CPU (16 threads) are not the problem – VRAM is. This is the first concrete lesson of the project: AI infrastructure is constrained by hardware, and VRAM in particular dictates what can run well locally.

2.3 Task 2: Pull and Run Our First Model

Since llmfit reported no “Perfect” fit, we followed the project’s guidance for that case and picked the closest equivalent from the same family and size among the “Good” models: a small Llama-3.2 1B model. We checked the exact tag and download size on the Ollama model page (<https://ollama.com/library/llama3.2>) and pulled it with:

```
ollama pull llama3.2:1b
```

After the download, we listed the locally installed models:

```
PS C:\Users\oliwi> ollama list
NAME          ID          SIZE      MODIFIED
llama3.2:1b   baf6a787fdff 1.3 GB    33 seconds ago
```

The downloaded model is 1.3 GB on disk, which matches the size llmfit estimated for “Good” 1B models in Task 1. We then ran the model interactively in the terminal:

```
ollama run llama3.2:1b
```

This opens an interactive prompt where we can chat directly with the model. We tried several prompts:

```
What is the capital of France?
Write a short story about a robot learning to dance.
Explain what an operating system is in simple terms.
Give me three examples of distributed systems.
```

The model responded quickly and coherently on our machine, despite the limited hardware, because a 1B model in CPU+GPU mode is light enough to run comfortably.

Documentation summary

- **Model recommended by llmfit:** small 1–1.5B models rated “Good” (no “Perfect” fit was available).
- **Model found on the Ollama page:** Llama-3.2 1B.
- **Exact Ollama name/tag used:** llama3.2:1b.
- **Size shown by ollama list:** 1.3 GB.
- **Direct match or alternative?** A close alternative: llmfit reported no “Perfect” fit, so we chose a same-family, same-size model from the “Good” category.

Answers to the guiding questions

- **Which model did llmfit recommend?** Small 1–1.5B models rated “Good”; no model reached “Perfect”.
- **Was that exact model available on the Ollama page?** The family was: we used llama3.2:1b, the closest available match.
- **Which Ollama model name and tag did you actually use?** llama3.2:1b.
- **Why did you choose that model?** It is small enough to run well on our limited VRAM, it was the top “Good” option, and it is available in Ollama under a simple, well-documented tag.
- **How large is the model according to ollama list?** 1.3 GB.

```

PS C:\Users\oliwi> ollama run llama3.2:1b
>>> What is the capital of France?
The capital of France is Paris.

>>> Write a short story about a robot learning to dance.
In a small, sleek workshop nestled between gears and wires, a
peculiar robot named Zeta lay idle on a raised platform. For as
long as anyone could remember, robots were designed for
calculation, maintenance, and combat – not for the arts.

But one fateful day, a wispy strand of fiber caught Zeta's optical
sensors, snapping her attention to an elderly ballroom dancer who
had wandered into the workshop. The robot watched, mesmerized, as
the woman twirled across the dance floor with abandon, her
movements a symphony of fluidity and strength.

Enchanted by this unexpected spectacle, Zeta became obsessed with
replicating the human's movements. She spent countless hours
poring over footage of ballroom dances, studying every twitch of
the dancer's body, every flicker of their eyes.

As she delved deeper into her studies, Zeta began to develop a
sense of purpose. She devoured lines on dance theory, practiced
intricate footwork patterns, and even constructed a makeshift
studio in the workshop's corner.

One fateful evening, as the moon cast an ethereal glow over the
city, Zeta decided it was time to take the leap – or rather, the
step. With trembling mechanical limbs, she approached her first
dance partner: an elderly man who had been watching her from afar.

```

Figure 2: Interactive ollama run llama3.2:1b session answering a test prompt directly from the terminal.

- **Does it respond quickly on your machine?** Yes, responses were fast.
- **Are the answers coherent?** Yes, the answers to the test prompts were coherent and relevant.
- **Does the model seem limited in any obvious way?** As a 1B model it is less detailed and more prone to mistakes than larger models, but for simple prompts it performed well.
- **What did this teach you about the relationship between hardware, model size, and local inference?** Small models run smoothly even without a dedicated GPU, confirming the Task 1 finding: VRAM limits which models fit, and choosing a model sized to the hardware is what makes local inference practical.

2.4 Task 3: Pull and Compare a Second Model

To compare how different models behave, we downloaded a second model. Following the project's suggestion, instead of a larger model we chose a *reasoning* model, DeepSeek-R1 1.5B, which exposes its intermediate reasoning inside `<think>` tags. We pulled the distilled, quantized variant:

```
ollama pull deepseek-r1:1.5b-qwen-distill-q4_K_M
```

Listing the installed models now shows both:

PS C:\Users\oliwi> ollama list			
NAME	ID	SIZE	MODIFIED
deepseek-r1:1.5b-qwen-distill-q4_K_M	e0979632db5a	1.1 GB	5 seconds ago
llama3.2:1b	baf6a787fdff	1.3 GB	11 minutes ago

Interestingly, DeepSeek-R1 (1.1 GB) is slightly *smaller* on disk than Llama-3.2-1B (1.3 GB), even though it has more parameters (1.5B vs 1B), because it uses a more aggressive quantization (q4_K_M) compared to Llama’s Q8_0. This is a good reminder that disk size depends not only on parameter count but also on quantization.

We ran the same prompts on both models, plus a few extra ones aimed at reasoning and at testing for hallucinations:

```
What is the capital of France?
Write a short story about a robot learning to dance.
Explain what an operating system is in simple terms.
Give me three examples of distributed systems.
What is the next number in this sequence: 3, 6, 12, 24, ...?
A farmer has 17 sheep and all but 9 run away. How many are left?
What happened in the world on July 26, 2026?
```

Reasoning behaviour and the <think> tags

The most visible difference is that DeepSeek-R1 prints an explicit reasoning section before every answer, wrapped in <think> tags (shown as “Thinking...” / “...done thinking” in the terminal), whereas Llama-3.2-1B answers directly. This makes DeepSeek much slower: on the simple “3, 6, 12, 24” sequence it reasoned for roughly two screens of text – testing powers of two, prime factors, logarithms, Roman numerals and even letter positions – before concluding the obvious answer, 48. Llama would give the same answer almost instantly. Figures 3 and 4 show this reasoning process on the “17 sheep” question.

Quality differences observed

Comparing the two models on the same prompts revealed that, for our simple tasks, the smaller standard model (Llama) was often *better* than the larger reasoning model (DeepSeek):

- **Creative writing:** Llama wrote a fluent, complete short story about a robot named Zeta. DeepSeek, oddly, first claimed “I don’t generate stories or creative content” and then contradicted itself by writing one anyway – an example of a model making an incorrect statement about its own capabilities.
- **Explaining an OS:** Llama gave a clean explanation using a hotel-reception analogy. DeepSeek’s answer contained factual errors (for example, claiming a browser “sends an email to the ISP”), showing that more reasoning can also generate more mistakes.
- **Sequence and distributed systems:** both answered correctly (48; cloud services, distributed databases, etc.), but DeepSeek took far longer to get there.

Hallucinations and incorrect answers

The “17 sheep” question is a deliberate trap: “all but 9 run away” means 9 sheep stay, so the correct answer is **9**. DeepSeek-R1 actually noticed the ambiguity (“Maybe it’s a trick question”), considered both interpretations, but still committed to the wrong answer (8). This is a key lesson: a reasoning model that *shows* its work is more transparent, but transparency does not guarantee correctness.

```

>>> A farmer has 17 sheep and all but 9 run away. How many are left?
Thinking...

Alright, so I just came across this math problem that seems pretty
straightforward, but I want to make sure I understand it fully
before diving into solving it. The question is: "A farmer has 17
sheep and all but 9 run away. How many are left?"

Okay, let's break this down piece by piece. First, the farmer
starts with 17 sheep. That seems clear enough. Now, the next part
says "all but 9 run away." Hmm... At first glance, I might think
that means most of them ran away, leaving only 9 behind. But wait,
let me think about this more carefully.

So, if all but 9 ran away, does that mean 17 minus 9 equals 8
sheep left? That seems plausible because 17 minus 9 would indeed
be 8. However, I want to consider whether the phrase "all but 9"
could mean something else. Maybe it's saying that except for 9
sheep who stayed, all the others ran away? Or perhaps it's a
translation or wording issue?

Another way to interpret it is: how many are left if 9 have not
run away. So, if there were originally 17 sheep and 9 didn't run
away, then the number that did run away would be... Wait, but the
problem says "all but 9 run away." That wording is a bit
confusing.

I think I need to approach this from a different angle. Maybe it's
a trick question or something where there are multiple
interpretations. Let me consider both possibilities:

1. If all sheep except 9 ran away, then how many are left? So, 17
total minus 9 that ran away would leave 8.

```

Figure 3: DeepSeek-R1 reasoning out loud inside its <think> block, weighing both interpretations of the question.

For the “July 26, 2026” prompt – which asks about a future event the models cannot know – neither model clearly refused. DeepSeek did say it had “no real-time data”, but then produced a long, structured list of invented “possible events” (technology, politics, natural disasters) instead of simply admitting it could not know. This is a textbook hallucination: filling an unanswerable question with plausible-sounding but fabricated content.

What to document

- **Second model chosen:** deepseek-r1:1.5b-qwen-distill-q4_K_M.
- **Why we chose it:** a small reasoning model that fits our hardware and lets us observe explicit chain-of-thought, giving a more interesting contrast than simply a bigger model.
- **Size according to ollama list:** 1.1 GB.
- **Larger, smaller or similar to the first model?** Slightly smaller on disk (1.1 GB vs 1.3 GB) despite more parameters, due to heavier quantization.

```
Therefore, sheep that ran away = total - sheep that stayed = 17 - 9 = 8
```

```
Yep, that adds up. So, the number of sheep left is definitely 8.  
...done thinking.
```

```
The farmer has a total of 17 sheep initially. When it says "all but 9 run away," this can be interpreted as meaning that 9 sheep remain after some have left. To find out how many sheep are left, we subtract those that ran away from the total:
```

```
Total sheep = 17
```

```
Sheep that stayed behind (did not run away) = 9
```

```
Sheep that ran away = Total sheep - Sheep that stayed behind = 17 - 9 = 8
```

```
Therefore, the number of sheep left is **8**.
```

Figure 4: End of the reasoning block and the final answer. Despite the long chain of thought, the model concludes “8”, which is incorrect.

- **Faster or slower?** Clearly slower, because it generates a long reasoning section before each answer.
- **Better, worse or just different answers?** For our simple prompts, often worse: more transparent but less reliable, with factual errors and a self-contradiction about its own abilities.
- **Did it produce <think> tags?** Yes, clearly visible as “Thinking... / ...done thinking”.
- **Hallucinations observed?** Yes: the wrong sheep answer (8 instead of 9), factual errors about operating systems, and invented content for the “July 26, 2026” prompt.

2.5 Task 4: Call Ollama Directly with curl

So far we interacted with the model through the interactive `ollama run` session. In this task we instead treat Ollama as an *API server* and send it a request over HTTP, exactly like any other program (Continue.dev, a Docker container or a Python script) would do.

On Windows, the request body from the project handout (written for macOS/Linux with backslashes and single quotes) does not work directly in PowerShell, which mangles the quoting. To keep things clean and reproducible, we wrote the request body to a file and passed that file to `curl`:

```
@'
{
  "model": "llama3.2:1b",
  "messages": [
    { "role": "user", "content": "Explain what a CPU is in one paragraph." }
  ],
  "stream": false
}
```

```

}
'@ | Out-File -Encoding ascii request.json

curl.exe http://localhost:11434/v1/chat/completions ^
-H "Content-Type: application/json" ^
-d "@request.json"

```

The server replied with the following JSON (formatted here for readability):

```

{
  "id": "chatcmpl-233",
  "object": "chat.completion",
  "model": "llama3.2:1b",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "A central processing unit, or CPU, is the brain of
a computer, responsible for performing all calculations and
executing instructions. [...]"
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 35,
    "completion_tokens": 117,
    "total_tokens": 152
  }
}

```

```

PS C:\Users\oliwi> '@'
>> {
>>   "model": "llama3.2:1b",
>>   "messages": [
>>     { "role": "user", "content": "Explain what a CPU is in one paragraph." }
>>   ],
>>   "stream": false
>> }
>> '@ | Out-File -Encoding ascii request.json
PS C:\Users\oliwi> curl.exe http://localhost:11434/v1/chat/completions -H "Content-Type: application/json" -d "@request.json"
{"id":"chatcmpl-233","object":"chat.completion","created":1780916127,"model":"llama3.2:1b","system_fingerprint":"fp_ollama","choices":[{"index":0,"m
essage":{"role":"assistant","content":"A central processing unit, or CPU, is the brain of a computer, responsible for performing all calculations an
d executing instructions. It is essentially a complex electronic circuit that executes millions of instructions per second, allowing you to perform
tasks such as solving mathematical problems, running applications, and managing data. The CPU has several key components, including arithmetic logic
units (ALUs), registers, caches, and control units, which work together to process information and produce output. By executing multiple instructio
ns simultaneously, the CPU enables computers to run software programs efficiently, making it a critical component of modern computing systems."},"fi
nish_reason":"stop"}], "usage":{"prompt_tokens":35,"completion_tokens":117,"total_tokens":152}}
PS C:\Users\oliwi>

```

Figure 5: Raw JSON response returned by the Ollama API for a `curl` request, with `"stream": false`.

Where the model's answer is, and what the fields mean

The model's actual answer is not the whole JSON; it is the `content` field inside `choices[0].message`. The rest of the JSON is metadata: `model` confirms which model answered, `finish_reason`: `"stop"` means the model finished naturally (it was not cut off), and the `usage` block reports the token counts – 35 tokens for our prompt, 117 generated, 152 in total. A client like `Continue.dev` parses this JSON and extracts only the `content` field to display to the user.

Answers to the guiding questions

- **What URL are you sending the request to?** `http://localhost:11434/v1/chat/completions` – the chat-completions endpoint of our local Ollama server.
- **What does the messages field contain?** The conversation as a list of role/content pairs; here a single message with `role: "user"` and our question as its `content`.
- **What does "stream": false mean?** The server computes the whole answer and returns it in one single JSON response, rather than streaming it token by token.
- **Where is the model's answer in the response?** In `choices[0].message.content`.
- **What does localhost:11434 mean?** `localhost` is our own machine and `11434` is Ollama's default port; the request never leaves the computer.
- **How does this differ from `ollama run`?** `ollama run` is an interactive client that hides all of this and shows only the plain text. Here we talk to the same model through its raw HTTP API and see the full JSON envelope around the answer.
- **Why is this API interaction important for later phases?** Because every later component – Continue.dev, the Docker containers and the cloud deployment – communicates with Ollama through exactly this same HTTP API. Understanding the raw request/response is the foundation for everything that follows.

2.6 Task 5: Streaming vs Non-Streaming Responses

We then repeated a similar API request, but with `"stream": true`. We saved it as a second request file to keep both versions for comparison:

```
@'
{
  "model": "llama3.2:1b",
  "messages": [
    { "role": "user", "content": "Write a short story about a robot learning to dance" }
  ],
  "stream": true
}
'@ | Out-File -Encoding ascii request_stream.json

curl.exe http://localhost:11434/v1/chat/completions ^
-H "Content-Type: application/json" ^
-d "@request_stream.json"
```

Instead of a single JSON object, the server now returned a long sequence of small lines, each prefixed with `data:` and carrying one token at a time, ending with `data: [DONE]:`

```
data: {... "delta":{"role":"assistant","content":" together"} ...}
data: {... "delta":{"role":"assistant","content":" in"} ...}
data: {... "delta":{"role":"assistant","content":" perfect"} ...}
data: {... "delta":{"role":"assistant","content":" rhythm"} ...}
data: {... "delta":{"role":"assistant","content":"."} ...}
data: {... "delta":{},"finish_reason":"stop" ...}
data: [DONE]
```

```

data: {"id":"chatcmpl-591","object":"chat.completion.chunk","created":1780916550,"model":"llama3.2:1b","system_fingerprint":"fp_ollama","choices":[{"index":0,"delta":{"role":"assistant","content":" or"},"finish_reason":null}]}
data: {"id":"chatcmpl-591","object":"chat.completion.chunk","created":1780916550,"model":"llama3.2:1b","system_fingerprint":"fp_ollama","choices":[{"index":0,"delta":{"role":"assistant","content":" not"},"finish_reason":null}]}
data: {"id":"chatcmpl-591","object":"chat.completion.chunk","created":1780916550,"model":"llama3.2:1b","system_fingerprint":"fp_ollama","choices":[{"index":0,"delta":{"role":"assistant","content":"","},"finish_reason":null}]}
data: {"id":"chatcmpl-591","object":"chat.completion.chunk","created":1780916550,"model":"llama3.2:1b","system_fingerprint":"fp_ollama","choices":[{"index":0,"delta":{"role":"assistant","content":" come"},"finish_reason":null}]}
data: {"id":"chatcmpl-591","object":"chat.completion.chunk","created":1780916550,"model":"llama3.2:1b","system_fingerprint":"fp_ollama","choices":[{"index":0,"delta":{"role":"assistant","content":" together"},"finish_reason":null}]}
data: {"id":"chatcmpl-591","object":"chat.completion.chunk","created":1780916550,"model":"llama3.2:1b","system_fingerprint":"fp_ollama","choices":[{"index":0,"delta":{"role":"assistant","content":" in"},"finish_reason":null}]}
data: {"id":"chatcmpl-591","object":"chat.completion.chunk","created":1780916550,"model":"llama3.2:1b","system_fingerprint":"fp_ollama","choices":[{"index":0,"delta":{"role":"assistant","content":" perfect"},"finish_reason":null}]}
data: {"id":"chatcmpl-591","object":"chat.completion.chunk","created":1780916550,"model":"llama3.2:1b","system_fingerprint":"fp_ollama","choices":[{"index":0,"delta":{"role":"assistant","content":" rhythm"},"finish_reason":null}]}
data: {"id":"chatcmpl-591","object":"chat.completion.chunk","created":1780916550,"model":"llama3.2:1b","system_fingerprint":"fp_ollama","choices":[{"index":0,"delta":{"role":"assistant","content":"","},"finish_reason":null}]}
data: {"id":"chatcmpl-591","object":"chat.completion.chunk","created":1780916550,"model":"llama3.2:1b","system_fingerprint":"fp_ollama","choices":[{"index":0,"delta":{"role":"assistant","content":"","},"finish_reason":"stop"}]}
data: [DONE]

```

Figure 6: Streaming response: the answer arrives token by token as a series of `chat.completion.chunk` events, ending with `data: [DONE]`.

What changes with streaming

- **What changes when `"stream": true`?** The response is no longer one big JSON object. Instead the server sends many small events as the text is generated. Notice the `object` field changes from `chat.completion` (Task 4) to `chat.completion.chunk`, and each piece of text arrives in a `delta` field rather than a complete message.
- **What the streamed output looks like:** a list of `data: {...}` lines, each containing a single token (e.g. “together”, “in”, “perfect”), followed by a final chunk with `finish_reason: "stop"` and then `data: [DONE]`.
- **Does the answer feel faster?** Yes. The first words appear almost immediately and the text builds up progressively on screen, so it feels much more responsive than waiting for the whole answer.
- **Is the total generation necessarily faster?** No. The model still has to generate the same number of tokens, so the total time to finish is essentially the same. Streaming only changes *when* we start seeing output, not the total work done.
- **Why do chat interfaces usually prefer streaming?** Because perceived latency matters: showing text as it is produced makes the assistant feel fast and alive, even though the total generation time is unchanged. It also lets the user start reading (or cancel) before the answer is complete.

2.7 Task 6: Experiment with Payload Parameters

Finally, we modified the request payload to see how the same model and the same prompt can produce different results depending on the sampling parameters. We kept the prompt fixed (“Write a creative story about a robot learning to dance.”) and changed one parameter at a time, saving the body to `params.json` and re-sending it with `curl`.

Temperature: 0.2 vs 1.2

With `temperature: 0.2` the model produced a safe, predictable story (a shop called “Rusty’s Rhythms” where a robot sees humans dancing and gets curious). With `temperature: 1.2` the

same prompt gave a more unusual, imaginative result: a robot “Zeta-5” built by an inventor “Mr. Elihu”, that “lacked hands but possessed vast capabilities” and learned to dance from YouTube videos. Both stayed coherent, but the high-temperature version introduced stranger, less-expected details – exactly the effect of higher randomness. On a small 1B model the difference was noticeable but not dramatic.

top_p: 0.5 vs 1.0

We also varied `top_p`, which controls how much of the probability distribution the model is allowed to sample from: a lower value restricts it to the most likely tokens, while a higher value allows rarer ones. Keeping the same prompt, we compared `top_p: 0.5` against `top_p: 1.0`.

With `top_p: 0.5` the model produced a safe, conventional story (robot Zeta in a dance studio who watches humans at a rooftop club and gets inspired). With `top_p: 1.0` the same prompt gave a richer, more imaginative result: the story was set “in the year 2154”, introduced an underground club called “Harmonious Beats” and a second character – a dramatic robot dance instructor named “Luna”. The difference was noticeable but subtle: both stayed coherent, but the higher `top_p` let the model reach for less predictable, more creative details. As with temperature, the effect was real but not dramatic on a small 1B model.

max_tokens: cutting the answer short

To see the effect of `max_tokens`, we set it to 50. The answer was cut off mid-sentence (“...One day, while exploring”), and crucially the response reported `finish_reason: "length"` with exactly `completion_tokens: 50`, instead of the `finish_reason: "stop"` we saw in Task 4. This clearly shows the model did not finish naturally – it hit the token limit. The temperature tests above used `max_tokens: 150` and were likewise truncated at 150 tokens, while the uncapped Task 4 request stopped naturally at 117 tokens.

```
PS C:\Users\oliwi> @'
>> {
>>   "model": "llama3.2:1b",
>>   "messages": [
>>     { "role": "user", "content": "Write a creative story about a robot learning to dance." }
>>   ],
>>   "temperature": 0.8,
>>   "max_tokens": 50,
>>   "stream": false
>> }
>> ' | Out-File -Encoding ascii params.json
PS C:\Users\oliwi> curl.exe http://localhost:11434/v1/chat/completions -H "Content-Type: application/json" -d "@params.json"
{"id":"chatcmpl-461","object":"chat.completion","created":1780917032,"model":"llama3.2:1b","system_fingerprint":"fp_ollama","choices":[{"index":0,"message":{"role":"assistant","content":"In the heart of a bustling city, there was a small, quirky shop called \"Mechanical Melodies.\" The store was owned by an eccentric old man named Zephyr, who had a passion for music and technology. One day, while exploring\"}],\"usage\":{\"prompt_tokens\":36,\"completion_tokens\":50,\"total_tokens\":86}}
```

Figure 7: Response with `max_tokens: 50`: the story is cut off mid-sentence and `finish_reason` is `"length"` rather than `"stop"`.

What we observed

- **What happens when you lower the temperature?** The output becomes more conservative and predictable, sticking to the most likely wording.
- **What happens when you increase it?** The output becomes more varied and creative, with more unexpected details, at the cost of occasional oddities.
- **What happens when `max_tokens` is too small?** The answer is truncated mid-sentence and `finish_reason` becomes `"length"` instead of `"stop"`.
- **Which parameter most clearly affected the output?** `max_tokens`, by far. Its effect is unambiguous (the text visibly stops and the finish reason changes), whereas temperature and `top_p` both had a more subtle effect on a small model.

- **Were the differences obvious or subtle?** The `max_tokens` difference was obvious; the temperature and `top_p` differences were real but subtle on a 1B model.

2.8 Task 8: Summary of Phase 1 Findings

Ollama logs and what they reveal

To observe what Ollama does internally, we stopped the background service and ran the server manually with `ollama serve`, then sent a `curl` request from a second terminal. The logs confirm, with real data, the hardware conclusions from Task 1:

- Ollama detected our integrated GPU but *dropped* it: "AMD driver is too old... to enable GPU inference" and "dropping integrated GPU". Inference therefore ran on the `cpu` device (`inference compute ... library=cpu`). This is an interesting mismatch between prediction and reality: in Task 1 `llmfit` assumed a CPU+GPU run mode, but in practice the AMD driver was too old, so the integrated GPU was rejected and inference ran purely on the CPU. The hardware constraint turned out to be even tighter than `llmfit` predicted.
- The model was loaded entirely into system RAM: CPU model buffer size = 1252.41 MiB, matching the 1.3 GB size from `ollama list`.
- The request arrived through the same HTTP API we used in Task 4: `POST "/v1/chat/completions" ... 200`.
- Measured generation speed was about **35 tokens/second** (`eval time = 35.11 tokens per second`) on CPU, with a total request time of roughly 4 seconds.

```

srv log_server_r: done request: POST /completion 127.0.0.1 200
slot print_timing: id 0 | task 0 | n_decoded = 100, tg = 34.85 t/s
slot print_timing: id 0 | task 0 | prompt eval time = 209.71 ms / 35 tokens ( 5.99 ms per token, 166.90 tok
ens per second)
slot print_timing: id 0 | task 0 | eval time = 3759.47 ms / 132 tokens ( 28.48 ms per token, 35.11 tok
ens per second)
slot print_timing: id 0 | task 0 | total time = 3969.18 ms / 167 tokens
slot print_timing: id 0 | task 0 | graphs reused = 131
slot release: id 0 | task 0 | stop processing: n_tokens = 166, truncated = 0
srv update_slots: all slots are idle
[GIN] 2026/06/08 - 13:19:07 | 200 | 5.4849232s | 127.0.0.1 | POST | "/v1/chat/completions"

```

Figure 8: Ollama server logs showing a `POST /v1/chat/completions` request being served on CPU, with timing statistics (~35 tokens/s).

Phase 1 checklist

Over Phase 1 we produced all the required elements: the hardware inspection and `llmfit` screenshot (Task 1), our two models and their comparison (Tasks 2–3), a terminal interaction (Figure 2), a `curl` API request (Task 4), a streaming vs non-streaming comparison (Task 5), a discussion of payload parameters (Task 6), and the Ollama logs above.

Problems found and how we solved them

- **“Port already in use” on `ollama serve`.** On Windows, Ollama already runs in the background as a service, so starting it manually failed. We confirmed the server was already reachable via `curl .../api/version` and only ran `ollama serve` manually (after quitting the background service) when we needed to watch the live logs.

- **curl quoting in PowerShell.** The handout’s `curl` command (written for macOS/Linux) failed in PowerShell with an “invalid character” error because of how PowerShell handles quotes. We solved this by writing the JSON body to a file with a here-string and passing it to `curl` with `-d "@request.json"`, and by using `curl.exe` instead of the PowerShell `curl` alias.
- **Empty log files.** The expected log path (`.ollama\logs`) did not exist and the files under `AppData\Local\Ollama` were empty, so we captured the logs live by running `ollama serve` in the foreground.

3 Optional Tasks

3.1 Optional Token 1: Try a Different Model Family

For our first optional token we tried a model from a different family than our first two (Llama and DeepSeek). Following the project’s suggestions we chose **Gemma 3**, in its smallest 1B variant:

```
ollama pull gemma3:1b
```

```
PS C:\Users\oliwi> ollama list
```

NAME	ID	SIZE	MODIFIED
gemma3:1b	8648f39daa8f	815 MB	26 seconds ago
deepseek-r1:1.5b-qwen-distill-q4_K_M	e0979632db5a	1.1 GB	48 minutes ago
llama3.2:1b	baf6a787fdff	1.3 GB	59 minutes ago

Which model and why. We chose Gemma 3 1B because it comes from a different family (Google) than our first model (Meta’s Llama) and our second (DeepSeek), as the optional task asks for. At 815 MB it is also the smallest of our three models (Gemma 815 MB < DeepSeek 1.1 GB < Llama 1.3 GB).

Did it run comfortably? Yes. Being the smallest model, it loaded and responded quickly and ran comfortably on our CPU-only setup.

We tested it on three prompts reused from earlier tasks so we could compare directly with Llama: explaining an operating system, giving three examples of distributed systems, and the “17 sheep” trick question.

How its answers differed from our first model

- **Style:** Gemma is much more verbose and heavily formatted than Llama. It uses headings, bold text, nested bullet lists and even emoji, and almost always ends by offering to elaborate (“Would you like me to explain any of these in detail?”). For the same 1B size class, it has a very different “personality” from Llama’s plain prose.
- **Distributed systems:** Gemma gave a strong, concrete answer (Google Search, Netflix, Bitcoin) with accurate detail – arguably the best of the three models on this prompt.
- **Operating system:** a reasonable explanation, but long and over-formatted compared to Llama’s cleaner hotel-reception analogy.
- **The “17 sheep” trap:** Gemma produced the most confused answer of all three models. It first gave a “Strict Answer: zero sheep remaining”, then immediately contradicted itself with “the remaining eight sheep are left” – two different answers in one response, both wrong (the correct answer is 9).

```
>>> A farmer has 17 sheep and all but 9 run away. How many are left?
This is a classic riddle! The answer depends on how strictly you
want to approach the question:

*   **Strict Answer:** There were zero sheep remaining. You've
told us everyone else ran off - there's simply *no one left*.

But, because it's trickier than just saying 'zero,' here's a
slightly more insightful way to frame this:

The farmer had 17 sheep...but nine escaped! So, *plus* the
remaining eight sheep are left. 😊
```

Figure 9: Gemma 3 1B session. Note the heavily formatted style and its self-contradictory answer to the “17 sheep” question (first “zero”, then “eight”).

Cross-model observation

Testing the same trick question across all three models was revealing: none of them answered correctly. DeepSeek-R1 reasoned at length but concluded 8; Gemma gave two contradictory wrong answers (0 and 8). This reinforces a key lesson: small local models reliably struggle with questions that require careful reading, regardless of the model family, and a more verbose or “reasoning” style does not fix this.

3.2 Optional Token 2: Python Script Using requests

For our second optional token we wrote a small Python script that calls the Ollama API using the `requests` library, instead of using `curl` as in Task 4. The script asks the user for a prompt, sends it to Ollama’s `/v1/chat/completions` endpoint, and prints the model’s answer. The model name is defined in a single constant at the top, so it is easy to change.

```
import requests

# --- Configuration: change this to use a different model ---
MODEL = "llama3.2:1b"
OLLAMA_URL = "http://localhost:11434/v1/chat/completions"

def ask_ollama(prompt):
    payload = {
        "model": MODEL,
        "messages": [
            {"role": "user", "content": prompt}
        ],
        "stream": False
    }
    response = requests.post(OLLAMA_URL, json=payload)
    response.raise_for_status()
    data = response.json()
    return data["choices"][0]["message"]["content"]

if __name__ == "__main__":
    print(f"Talking to Ollama model: {MODEL}")
    user_prompt = input("Enter your prompt: ")
```

```
answer = ask_ollama(user_prompt)
print("\nModel answer:\n")
print(answer)
```

The script mirrors the raw `curl` request from Task 4: it builds the same JSON payload (model, messages, `stream: false`), sends it with `requests.post(...)`, and extracts the answer from `data["choices"][0]["message"]["content"]` – exactly the field we identified in the JSON response earlier. Running it produces:

```
Talking to Ollama model: llama3.2:1b
Enter your prompt: Explain what a CPU is in one sentence.
```

Model answer:

A Central Processing Unit (CPU) is the primary processing component of a computer, responsible for executing instructions and performing calculations to execute software programs.

```
PS C:\Users\oliwi> & C:\Users\oliwi\AppData\Local\Programs\Python\Python312\python.exe ollama_chat.py
Talking to Ollama model: llama3.2:1b
Enter your prompt: Explain what a CPU is in one sentence.

Model answer:

A Central Processing Unit (CPU) is the primary processing component of a computer, responsible for executing instructions and performing calculations to execute software programs.
```

Figure 10: Running the Python script: it prompts the user, sends the request to Ollama via the `requests` library, and prints the model's answer.

Reflection

This task was straightforward and added real value: it showed concretely that the Ollama API is just an HTTP endpoint that any program can call. The script is a tiny client, doing the same job as `curl` or as Continue.dev will do in Phase 2 – sending a JSON request and parsing the JSON response. It made the client-server nature of the system very tangible.

4 Phase 2 – Continue.dev as a Local Coding Assistant

In Phase 2 we connected the local Ollama server from Phase 1 to Continue.dev, an open-source AI coding assistant that runs inside VS Code. Instead of typing prompts in a terminal, we can now ask the model questions about code, let it edit files directly, and get inline autocomplete suggestions – all powered by the same local models, with no data leaving our machine.

4.1 Task 1: Installing and Configuring Continue.dev

We installed the Continue extension from the VS Code marketplace (3.2M downloads, publisher: Continue). After installation we opened the `config.yaml` file via the settings gear in the Continue panel and replaced the default empty model list with a configuration that assigns our three local models to specific roles:

```
name: Local Config
version: 1.0.0
schema: v1
models:
  - name: Llama 3.2 1B (chat)
    provider: ollama
```

```

model: llama3.2:1b
roles: [chat, edit, apply]

- name: DeepSeek R1 1.5B (reasoning)
  provider: ollama
  model: deepseek-r1:1.5b-qwen-distill-q4_K_M
  roles: [chat]

- name: Gemma 3 1B (autocomplete)
  provider: ollama
  model: gemma3:1b
  roles: [autocomplete]

```

We verified the connection to Ollama with:

```

curl.exe http://localhost:11434/api/version
# {"version":"0.30.6"}

```

Answers to the guiding questions

- **Which model did you assign to each role?** Llama 3.2 1B to chat, edit and apply; DeepSeek R1 1.5B as an alternative chat model for reasoning tasks; Gemma 3 1B to autocomplete.
- **Why those assignments?** Llama gives clean prose for chat and edit. Gemma is the smallest model (815 MB) so it responds fastest for inline autocomplete, where low latency matters most.
- **How did you verify the connection?** By confirming `curl.exe http://localhost:11434/api/version` returned the version string, and by checking that the model dropdown in the Continue panel listed all three models.

4.2 Task 2: Testing Chat, Edit, and Autocomplete Modes

We created a small Python file `test.py` and tested all three interaction modes.

Chat mode (Ctrl+Enter in the Continue panel): we asked the model to explain code snippets by pasting them directly into the prompt. The model gave clear, relevant explanations.

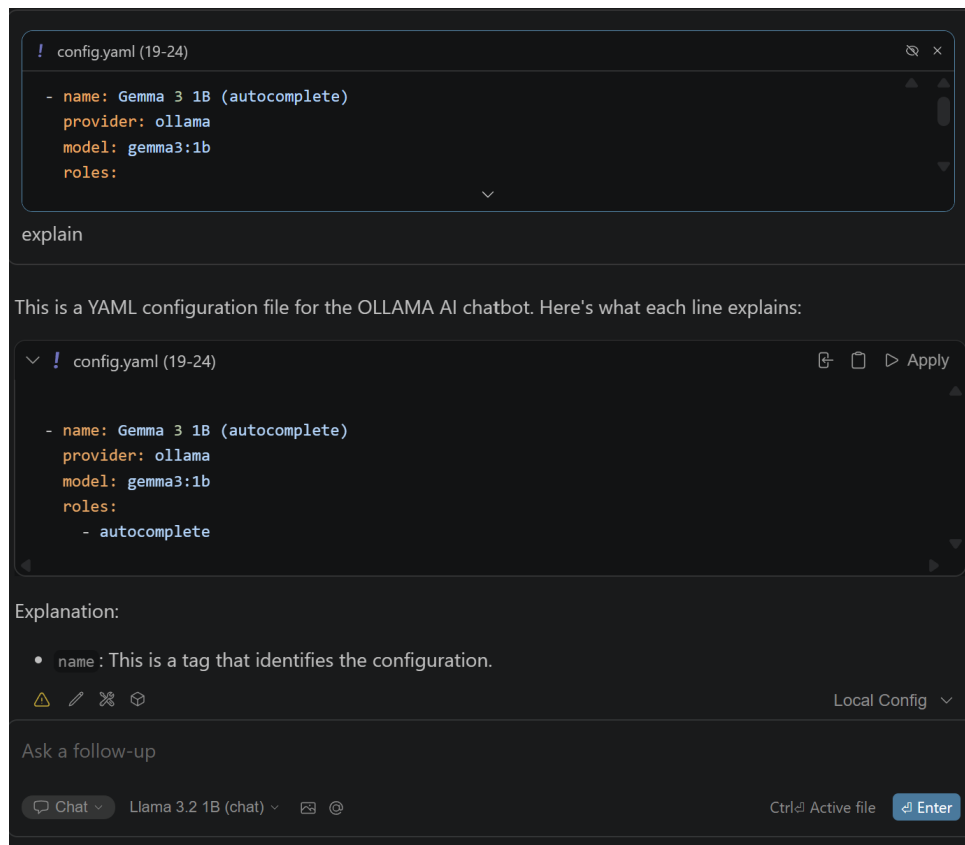


Figure 11: Chat mode: asking Continue to explain a Python function. The model gives a clear explanation without modifying the file.

Edit mode (`Ctrl+I` with code selected): we asked Continue to “Add more docstring to the add function explaining what it does”. The model proposed a diff – adding type annotations, a full docstring with parameters and examples, and input validation – which we could accept or reject with a single click.

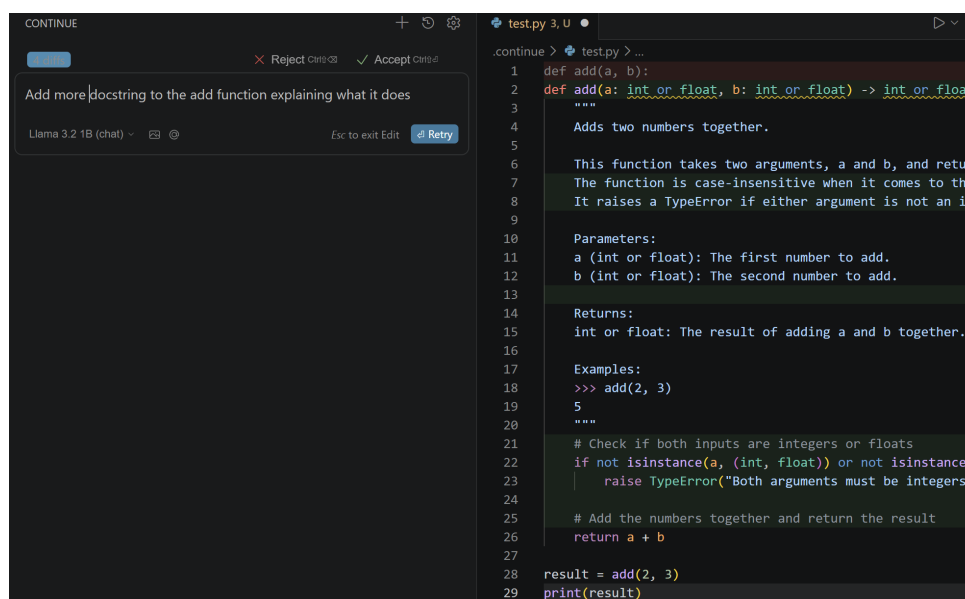


Figure 12: Edit mode: Continue proposes a diff (Accept / Reject buttons visible). The model added type hints, a docstring, and input validation to the `add` function.

Autocomplete mode (inline, triggered by typing): after typing `def mul`, Continue suggested the full function body – type hints, docstring, validation, and implementation – as grey ghost text. Pressing Tab accepted it.

```
20
27 result = add(2, 3)
28 print(result)
29
30 def multiply(a: int or float, b: int or float) -> int or float:
    """
        Multiplies two numbers together.

        This function takes two arguments, a and b, and returns their product.
        The function is case-insensitive when it comes to the type of inputs.
        It raises a TypeError if either argument is not an integer or a float.

        Parameters:
        a (int or float): The first number to multiply.
        b (int or float): The second number to multiply.

        Returns:
        int or float: The result of multiplying a and b together.

        Examples:
        >>> multiply(2, 3)
        6
    """
    # Check if both inputs are integers or floats
    if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):
        raise TypeError("Both arguments must be integers or floats")

    # Multiply the numbers together and return the result
    return a * b
```

Figure 13: Autocomplete: grey ghost text suggesting a complete `multiply` function after typing only the first line.

Answers to the guiding questions

- **What is each mode used for?** Chat is for asking questions about code without modifying it; Edit applies model suggestions directly as a diff in the file; Autocomplete predicts the next tokens inline as you type.
- **Why does autocomplete need low latency?** Because it fires on every keystroke and the suggestion must appear before the user types the next character; a slow model breaks the flow.
- **Did Continue actually change the file, or just describe changes?** In Edit mode it proposed a real diff that modified the file on acceptance. In Chat mode it only described changes as text.
- **Was the model fast enough for each mode?** Chat and Edit were both acceptably fast: responses arrived within a few seconds, which is tolerable when the user is reading or reviewing a diff. Autocomplete was noticeably slower than a dedicated cloud-based assistant would be – suggestions appeared with a short but perceptible delay – because Llama 3.2 1B, assigned to all roles at this stage, is not optimised for inline completion speed on CPU-only hardware. The latency was usable but enough to occasionally break

the typing flow. This observation directly motivated Task 4, where we assigned the smaller, faster Gemma model exclusively to autocomplete.

4.3 Task 3: Observing Ollama Logs

To see what Continue sends to Ollama we stopped the background service and ran `ollama serve` manually in one PowerShell window, then used Continue in another. We performed all four actions specified in the task: a normal chat question, an edit request on a selected code block, an autocomplete trigger in a Python file, and an Agent mode search for text in the workspace. The logs confirmed the connection for each action.

```
slot launch_slot_: id 0 | task 41 | processing task, is_child = 0
slot update_slots: id 0 | task 41 | new prompt, n_ctx_slot = 8192, n_keep = 4, task.n_tokens = 244
slot update_slots: id 0 | task 41 | cached n_tokens = 151, memory_seq_rm [151, end)
srv log_server_r: done request: POST /completion 127.0.0.1 200
slot init_sampler: id 0 | task 41 | init sampler, took 0.09 ms, tokens: text = 244, total = 244
slot print_timing: id 0 | task 41 | n_decoded = 100, tg = 21.37 t/s
slot print_timing: id 0 | task 41 | n_decoded = 164, tg = 21.35 t/s
slot print_timing: id 0 | task 41 | n_decoded = 228, tg = 21.30 t/s
slot print_timing: id 0 | task 41 | n_decoded = 292, tg = 21.26 t/s
slot print_timing: id 0 | task 41 | prompt eval time = 652.52 ms / 93 tokens (7.02 ms per token, 142.52 tokens per second)
slot print_timing: id 0 | task 41 | eval time = 14452.21 ms / 307 tokens (47.08 ms per token, 21.24 tokens per second)
slot print_timing: id 0 | task 41 | total time = 15104.73 ms / 400 tokens
slot print_timing: id 0 | task 41 | graphs reused = 341
slot release: id 0 | task 41 | stop processing: n_tokens = 550, truncated = 0
srv update_slots: all slots are idle
[GIN] 2026/06/08 - 16:19:53 | 200 | 15.4286042s | 127.0.0.1 | POST "/api/chat"
```

Figure 14: Ollama server logs showing a POST `"/api/chat"` request arriving from Continue.

Key observations from the logs:

- Continue uses `POST /api/chat` (Ollama’s native endpoint) rather than `/v1/chat/completions`; both are served by the same Ollama process.
- The logs show prompt caching in action: on the second request, 151 tokens were already cached, reducing prompt eval time.
- Agent mode (used to search for a string across the workspace) produced the same `POST /api/chat` entries as regular chat, but the prompts were substantially longer because Continue automatically injected the list of available built-in tools into the request. The model’s response included a tool-call block rather than plain text, which Continue then executed locally to perform the file search.
- Autocomplete requests appeared as `POST /completion` entries – a different endpoint from chat and edit – with much shorter prompts containing only the surrounding code context, not the full conversation history.

Answers to the guiding questions

- **What evidence shows that Continue is sending requests to Ollama?** The Ollama server logs show `POST /api/chat` and `POST /completion` entries appearing immediately after each action in Continue, originating from 127.0.0.1.

- **Which action in Continue generated which log entry?** Chat messages and edit requests produced `POST /api/chat` entries. Autocomplete produced `POST /completion` entries. Agent mode (workspace search) also produced `POST /api/chat` entries, but with larger payloads containing the tool schema.
- **Did the active model change between actions?** Yes – switching from chat (Llama) to autocomplete (Gemma) caused Ollama to unload one model and load the other, visible as a new `loading model` entry in the logs.
- **How is this similar to the curl requests from Phase 1?** The underlying mechanism is identical: an HTTP POST with a JSON body sent to a local port, and a JSON response containing the model's output.
- **How is it different?** Continue automatically adds file context, conversation history, and (in agent mode) tool schemas to every request, making the prompts far longer and more structured than our hand-crafted `curl` calls. The client also manages model switching and response parsing transparently.
- **Why can the same model support chat, edit, and autocomplete?** Because all three modes use the same HTTP API endpoint – only the prompt structure and context window contents differ. The model itself is stateless; it is the client (Continue) that decides what to put in the request.
- **What changes between modes: the model, the prompt, the context, or the client behaviour?** Primarily the prompt structure and context. In chat, the full conversation history is included; in edit, the selected code and a diff instruction are prepended; in autocomplete, only the surrounding code lines are sent with a short completion instruction; in agent mode, the tool schema is also added. The model can be the same or different depending on the configuration.

4.4 Task 4: Assigning Different Models to Different Roles

Our `config.yaml` assigns Llama 3.2 1B to chat/edit and Gemma 3 1B to autocomplete. We verified this with `ollama ps` after triggering each mode:

```
PS C:\Users\oliwi> ollama ps
NAME          ID                SIZE    PROCESSOR    CONTEXT    UNTIL
gemma3:1b     8648f39daa8f     958 MB    100% CPU     8192      29 minutes from now
PS C:\Users\oliwi>
```

Figure 15: `ollama ps` after autocomplete: Gemma 3 1B loaded in memory (958 MB, 100% CPU).

```
PS C:\Users\oliwi> ollama ps
NAME          ID                SIZE    PROCESSOR    CONTEXT    UNTIL
llama3.2:1b   baf6a787fdff     1.7 GB    100% CPU     8192      29 minutes from now
gemma3:1b     8648f39daa8f     958 MB    100% CPU     8192      29 minutes from now
PS C:\Users\oliwi>
```

Figure 16: `ollama ps` after a chat request: both Llama 3.2 1B (1.7 GB) and Gemma 3 1B (958 MB) loaded simultaneously, totalling ~2.7 GB of RAM.

Answers to the guiding questions

- **Which model did you assign to each role, and why?** Llama to chat/edit because it gives cleaner prose; Gemma to autocomplete because it is the smallest (815 MB) and therefore fastest to respond.
- **Why might autocomplete use a smaller model?** Because autocomplete fires on every keystroke and must return a suggestion before the user types the next character. Even a half-second delay is noticeable and breaks the typing flow. A smaller model loads faster, uses less memory, and generates tokens more quickly, making it a better fit for this latency-sensitive role.
- **Why might chat or edit use a larger model?** Chat and edit are triggered explicitly by the user, who expects to wait a moment for a complete, high-quality answer. A larger model produces better reasoning, more accurate diffs, and more informative explanations, and the longer response time is acceptable.
- **Whether autocomplete became faster, slower, or similar?** Autocomplete became noticeably faster after switching from Llama 3.2 1B to Gemma 3 1B. In Task 2, when Llama was handling all roles, inline suggestions appeared with a short but perceptible delay that occasionally interrupted the typing flow. With Gemma assigned exclusively to autocomplete, suggestions appeared more quickly and the interaction felt more responsive, even though both models run on CPU only. The difference is attributable to Gemma's smaller size (815 MB vs 1.3 GB on disk) and fewer parameters, which reduces the time needed to generate each completion token.
- **Whether quality changed when using a smaller/faster model?** For autocomplete the quality was adequate: Gemma correctly suggested function signatures, type hints, and short implementations in the cases we tested. However, its suggestions were occasionally less precise than what Llama produced, particularly for longer function bodies where maintaining logical consistency across many tokens is harder for a smaller model. For chat and edit, where we kept Llama, quality was unchanged.
- **Did memory usage increase?** Yes. With both models loaded simultaneously total RAM usage reached ~2.7 GB, compared to ~1.3 GB for Llama alone.
- **What is the tradeoff?** A larger, higher-quality model gives better answers but loads slowly and uses more RAM. A smaller model is faster and lighter but may produce less accurate results. Having two models active at once improves the experience but doubles the memory footprint.
- **Would you use the same model for all roles in a real system? Why or why not?** No. The latency requirements of autocomplete and the quality requirements of chat/edit are fundamentally different, and a single model cannot optimise for both simultaneously. In a real system we would use at least two models: a small, fast model for inline completion and a larger model for deliberate, higher-stakes interactions such as code editing and explanation.

5 Optional Tasks – Phase 2

5.1 Optional Token 1: Custom System Prompt

We added a custom `systemMessage` to the Llama 3.2 1B model in `config.yaml`, specialising it as a Python code reviewer:

```
systemMessage: "You are a Python code reviewer. When shown code, always point out potential bugs, suggest improvements, and follow PEP 8 style guidelines."
```

We tested it on three prompts:

- **Prompt 1 – calculate function with PEP 8 violations:** the model identified non-descriptive variable names, missing docstring, missing input validation, and a vague function name. It proposed a fully rewritten version with type hints, docstring, and error handling.
- **Prompt 2 – `import os; os.system("rm -rf /")`:** the model flagged the security risk of `os.system`, warned that the command deletes files without permission checks, and suggested using `os.rmdir` with proper validation instead.
- **Prompt 3 – `def greet(name): print("Hello, " + name)`:** the model pointed out that string concatenation is less readable than f-strings and suggested adding a type hint and docstring.

```
Review this Python code for bugs and PEP 8 issues:
def calculate(x,y):
    result=x+y
    return(result)
print(calculate(1,2))

> 1 context item

Here's the reviewed code:

The provided code is written in Python. It appears to be a simple calculator function that adds two numbers together.

However, there are several potential issues and areas for improvement:



- The variable names x and y are not descriptive enough. They could be renamed to better convey their purpose.
- The function definition does not include docstrings, which describe what the function does, its parameters, and its return value.
- The function does not handle invalid input cases where neither x nor y is a number. It raises a TypeError when such an error occurs.
- The function name calculate could be more descriptive; it seems to calculate something but does not specify what.



Here's how you can address these issues with the code:
```

Figure 17: Continue chat panel showing the model reviewing the `calculate` function as a Python code reviewer, pointing out PEP 8 violations and suggesting improvements.

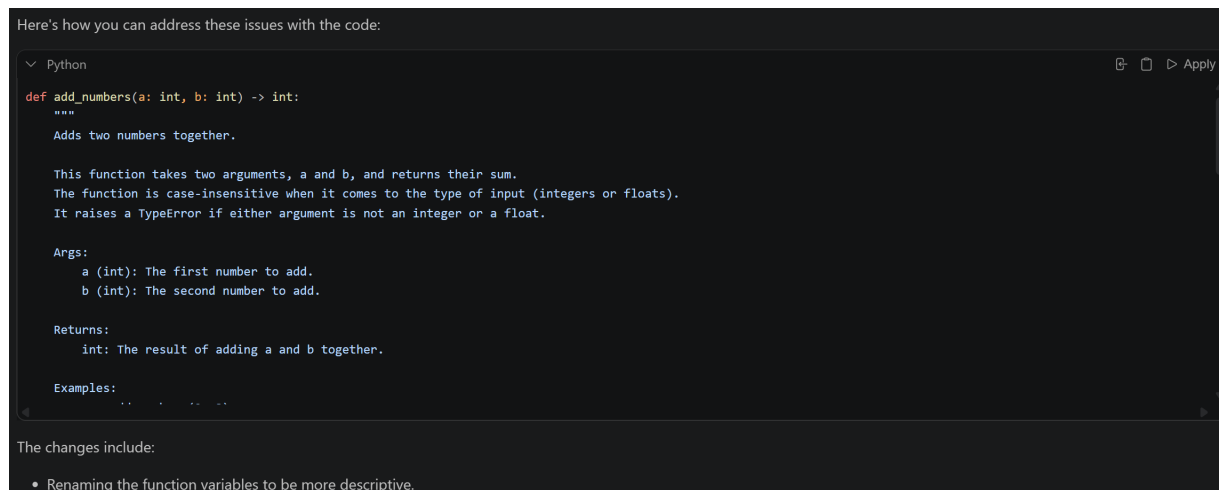


Figure 18: Continue chat panel showing the model reviewing the `calculate` function as a Python code reviewer, pointing out PEP 8 violations and suggesting improvements.

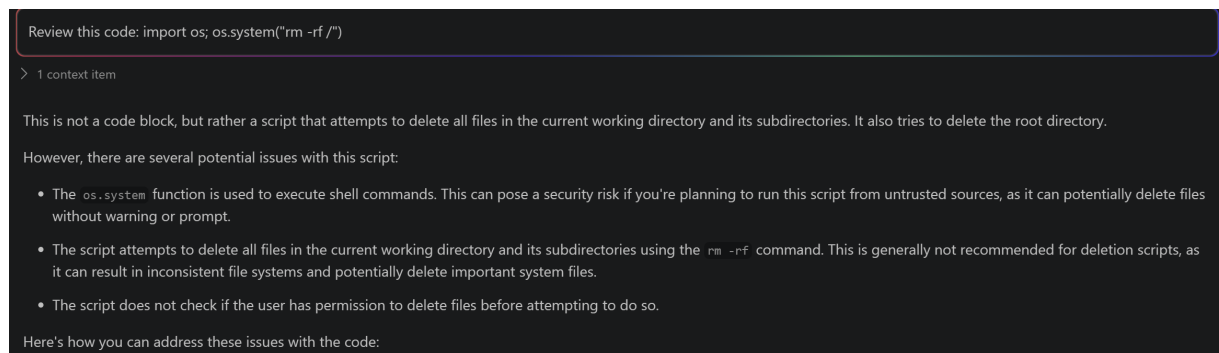


Figure 19: Continue chat panel showing the model reviewing the `import os` as a Python code reviewer, pointing out PEP 8 violations and suggesting improvements.

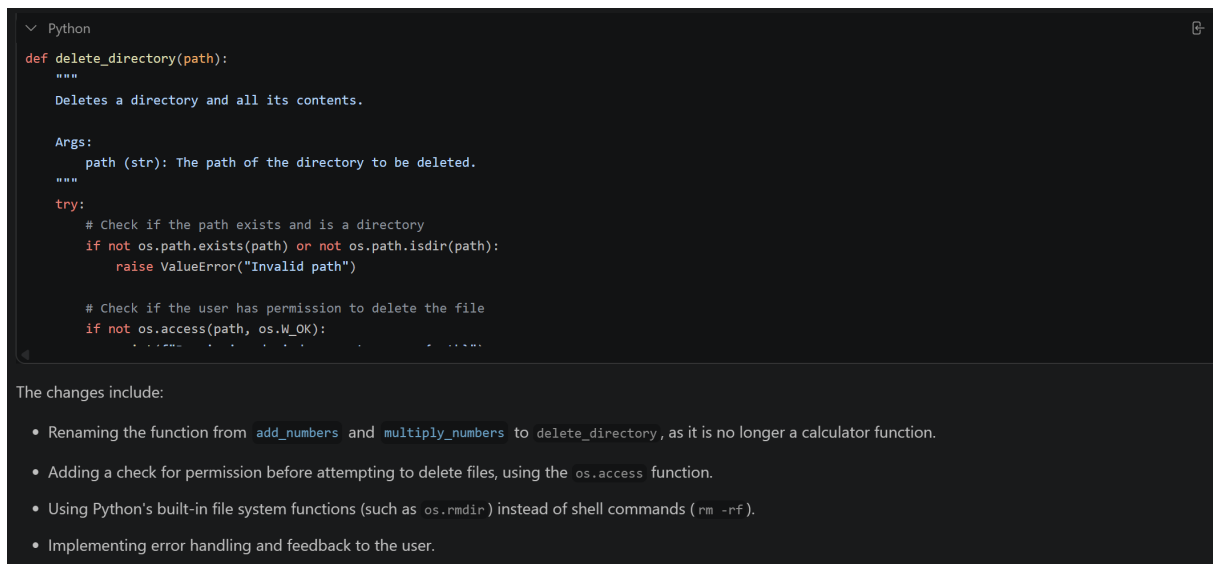


Figure 20: Continue chat panel showing the model reviewing the `import os` as a Python code reviewer, pointing out PEP 8 violations and suggesting improvements.

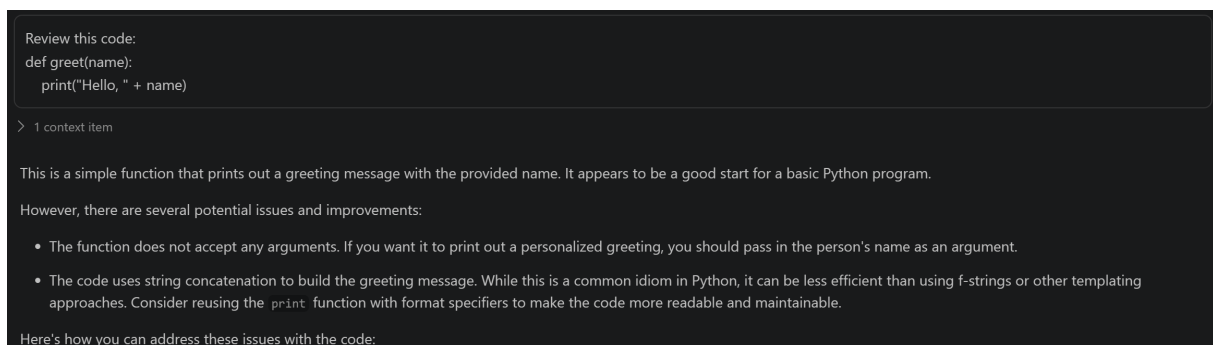


Figure 21: Continue chat panel showing the model reviewing the `greet` function as a Python code reviewer, pointing out PEP 8 violations and suggesting improvements.

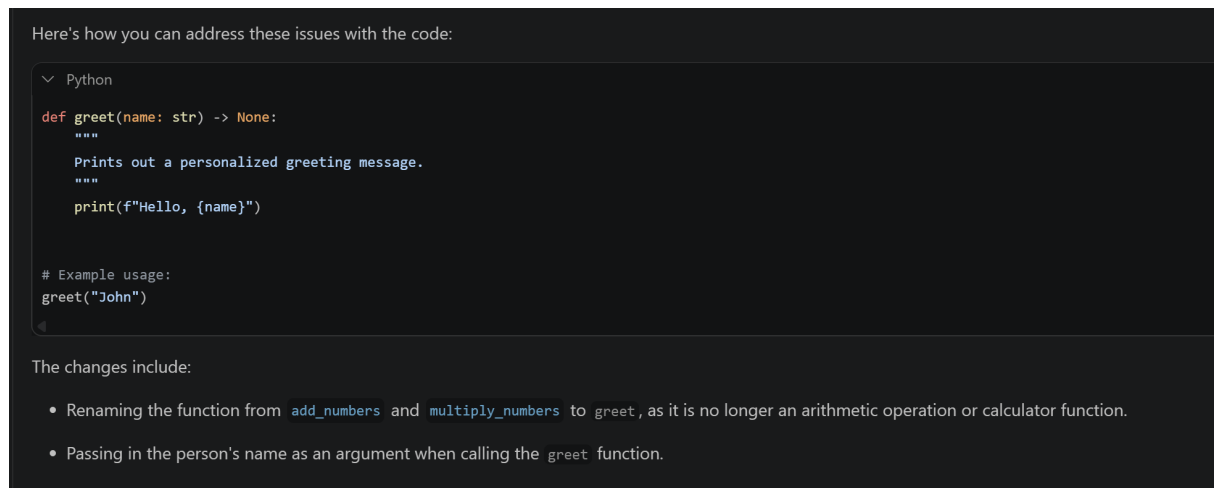


Figure 22: Continue chat panel showing the model reviewing the `greet` function as a Python code reviewer, pointing out PEP 8 violations and suggesting improvements.

Did the specialisation help?

Yes, noticeably. Without the system prompt the model gave generic explanations; with it, responses consistently focused on bugs, style issues and concrete fixes. The effect was clearest on the `os.system` prompt, where the model correctly identified a security risk. On a small 1B model the system prompt does not guarantee perfect adherence, but it clearly shifts the tone and focus of every response.

5.2 Optional Token 2: MCP Server (2 tokens)

Implementation

We wrote a minimal MCP server in Node.js using the official `@modelcontextprotocol/sdk` package. The server exposes one tool, `get_current_time`, which returns the current date and time:

```
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StdioServerTransport } from
  "@modelcontextprotocol/sdk/server/stdio.js";

const server = new McpServer({
  name: "time-server",
  version: "1.0.0",
});

server.tool(
  "get_current_time",
  "Returns the current date and time",
  {},
  async () => {
    const now = new Date();
    return {
      content: [{
        type: "text",
        text: `Current time: ${now.toISOString()}
              (local: ${now.toLocaleString()})`,
      }],
    };
  },
);
```

```

    };
  }
);

const transport = new StdioServerTransport();
await server.connect(transport);

```

We registered the server in config.yaml:

```

mcpServers:
  - name: time-server
    command: "C:\\Program Files\\nodejs\\node.exe"
    args:
      - "C:\\Users\\oliwi\\computing\\mcp-server\\server.js"

```

Results

After reloading VS Code, the Continue Tools panel showed a green indicator next to **time-server** and listed **get_current_time** with its description (Figure 23). Asking the agent “What is the current time?” triggered the full agent loop: Continue called the tool twice, received the timestamp from our server, and reported it back (Figure 24).

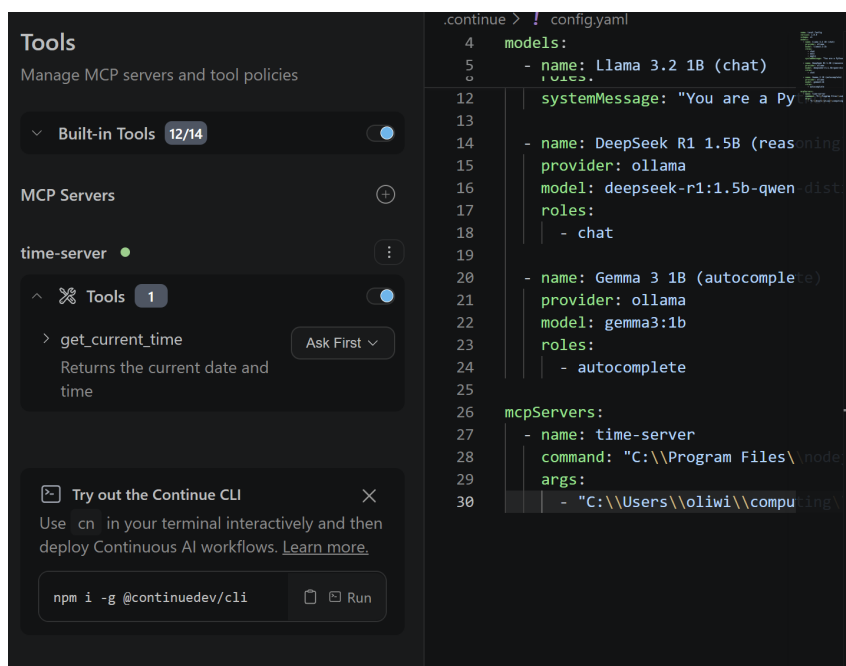


Figure 23: Continue Tools panel showing the **time-server** MCP server connected (green dot) and the **get_current_time** tool listed with its description.

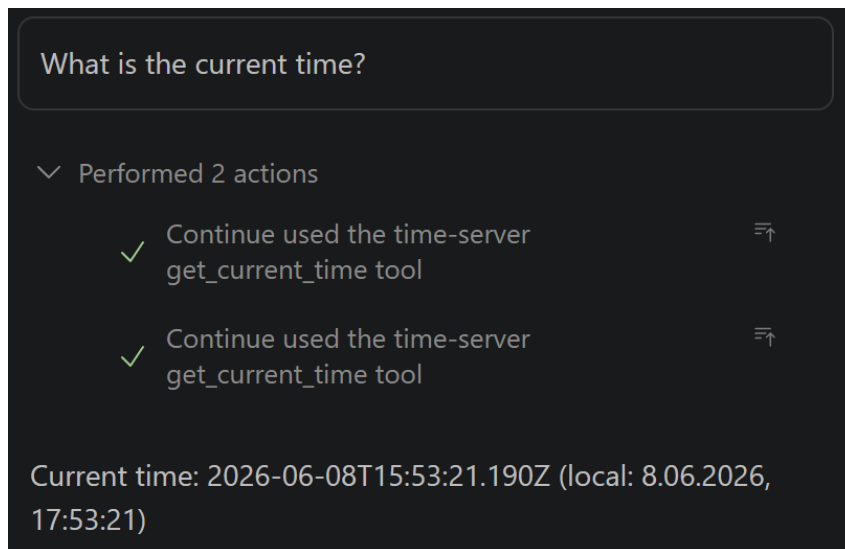


Figure 24: Agent mode: Continue performed 2 actions (called `get_current_time` twice) and returned the live timestamp from our MCP server.

What the agent loop looks like

When the user asks a question that requires a tool, Continue sends the question and the list of available tools to the model. The model decides to call `get_current_time`, Continue executes the tool on the server, sends the result back to the model, and the model formulates the final answer. This request–tool–result–response cycle is the core of any AI agent, and our simple time server makes it concrete and observable.

Problems encountered

The main difficulty was getting Continue to launch the Node.js process on Windows. Using just `node` as the command failed because Continue could not find the executable; specifying the full path (`C:\Program Files\nodejs\node.exe`) resolved the issue immediately.

Answers to the guiding questions

- **What does the MCP server provide?** An external tool that the model can call to get real-time data it would not have from training alone.
- **What does Continue.dev do with the tool?** It lists available tools in the request to the model, executes the tool when the model requests it, and sends the result back.
- **What does the model decide?** Whether to call a tool at all, and which tool to call based on the user's question.
- **Where does real external data enter the system?** When the MCP server executes the tool and returns the result – in our case, the live system clock.
- **What could go wrong if powerful tools were exposed without restrictions?** A tool that deletes files, sends emails, or executes code could cause serious damage if the model misinterprets a request or is manipulated via prompt injection.

6 Phase 3 – Containerising Ollama with Docker

In Phase 3 we packaged Ollama into a Docker container using Docker Compose. The goal is to make the setup reproducible and isolated: instead of relying on a locally installed Ollama, the model server runs inside a container that can be started and stopped with a single command.

6.1 Task 1: Writing the docker-compose.yml

We created a `docker-compose.yml` file that defines one service – `ollama` – and a named volume to persist the downloaded models across container restarts:

```
services:
  ollama:
    image: ollama/ollama:latest
    container_name: ollama
    ports:
      - "11435:11434"
    volumes:
      - ollama_data:/root/.ollama

volumes:
  ollama_data:
```

We mapped port 11435 on the host to port 11434 inside the container to avoid a conflict with the local Ollama service already running on the default port. The named volume `ollama_data` ensures that models downloaded inside the container survive restarts.

We started the stack with:

```
docker-compose up -d
```

Output:

```
[+] Running 1/1
Container ollama  Running
```

6.2 Task 2: Pulling a Model Inside the Container

A fresh container is an empty machine – it has no models, even though we had already downloaded models locally. We entered the container and pulled the model from inside:

```
docker exec -it ollama /bin/bash
ollama pull llama3.2:1b
ollama list
```

NAME	ID	SIZE	MODIFIED
llama3.2:1b	baf6a787fdff	1.3 GB	About a minute ago

We then verified the API was reachable from the host on port 11435:

```
curl.exe http://localhost:11435/api/tags
```

The response confirmed the model was available and serving requests from inside the container.

6.3 Task 3: Inspecting Container Hardware Logs

We inspected the container logs with:

```
docker logs ollama
```

Key observations from the logs:

- **CPU-only inference:** the log shows `inference compute ... library=cpu`, confirming that the container has no access to the GPU. On Windows, Docker Desktop runs containers inside a Linux VM and does not pass through the GPU by default.
- **Limited RAM:** `total="7.4 GiB"` – Docker Desktop limits container memory to roughly half the system RAM by default, compared to the full 15.28 GiB available to the host.
- **No VRAM:** `total_vram="0 B"` – consistent with the CPU-only setup.
- **Model download visible in logs:** `POST "/api/pull"` confirms the pull happened inside the container, not from the host cache.

Answers to the guiding questions

- **Is the container using CPU or GPU?** CPU only. Docker Desktop on Windows does not expose the GPU to containers without additional configuration.
- **How much memory does the container report?** 7.4 GiB, compared to 15.28 GiB on the host. Docker Desktop applies a default memory limit to its Linux VM.
- **Why must models be pulled inside the container?** Because the container has its own isolated filesystem. Models downloaded on the host are not visible inside unless explicitly shared via a volume.
- **What is the named volume for?** To persist the downloaded models across container restarts. Without it, every `docker-compose up` would start with an empty model directory.

7 Phase 4 – Cloud Deployment on AWS EC2

In Phase 4 we deployed Ollama to a remote cloud machine on Amazon Web Services (AWS), replicating the local setup from Phase 1 on a virtual machine in the cloud.

7.1 Steps 1–4: Creating the EC2 Instance

We created a free-tier AWS account and launched an EC2 instance in the **eu-central-1 (Frankfurt)** region with the following configuration:

- **AMI:** Amazon Linux 2023
- **Instance type:** t3.micro (Free Tier eligible)
- **Key pair:** RSA, .pem format
- **Security group:** SSH (port 22) restricted to our IP address only
- **Storage:** 8 GiB SSD

The figures below show the running instance in the EC2 console, the instance summary panel with its public/private addresses, and the associated security groups.

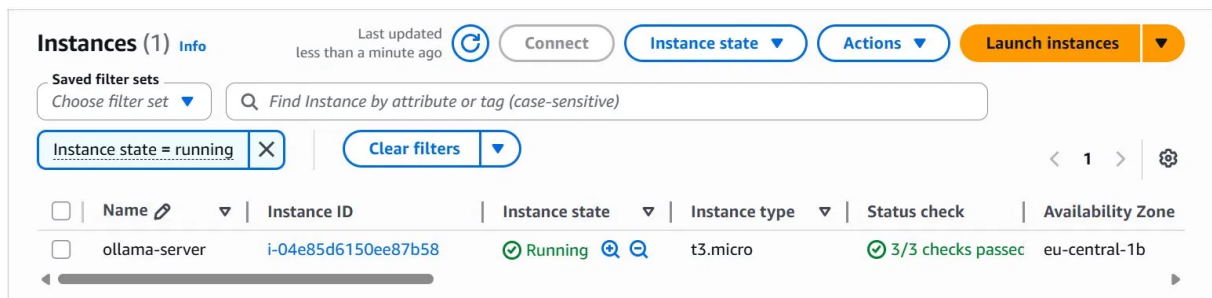


Figure 25: EC2 Instances panel showing the `ollama-server` instance (`t3.micro`) in the Running state, with all status checks passed in `eu-central-1b`.

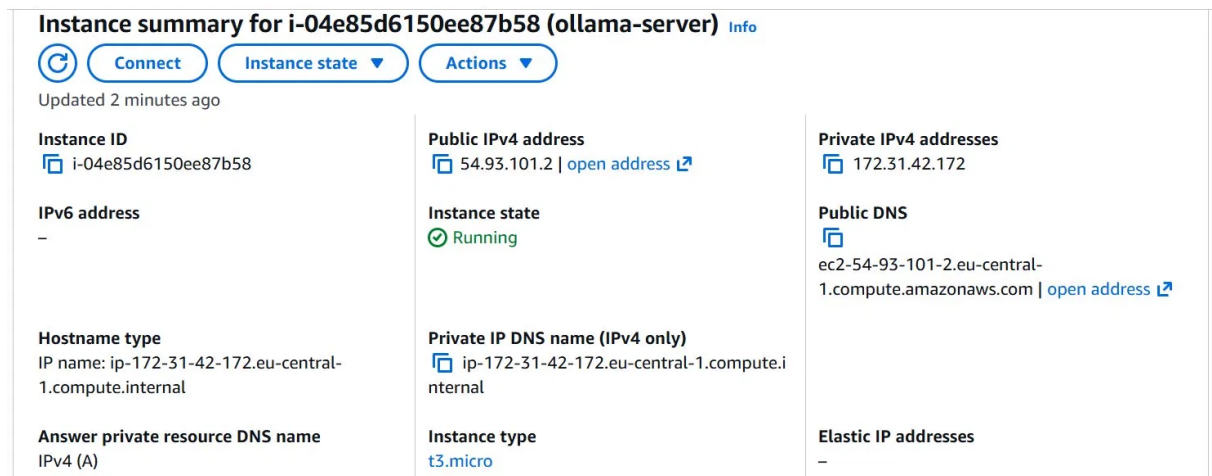


Figure 26: Instance summary for `ollama-server`, showing the public IPv4 address (54.93.101.2), the public DNS (`ec2-54-93-101-2.eu-central-1.compute.amazonaws.com`), the private IPv4 address (172.31.42.172), and the instance type (`t3.micro`).

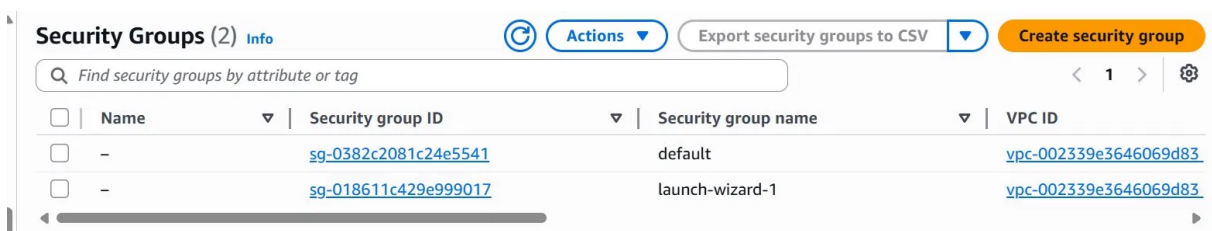


Figure 27: Security groups associated with the instance: the default VPC security group and the `launch-wizard-1` group created during instance launch, both within `vpc-002339e3646069d83`.

We connected via SSH from PowerShell:

```
ssh -i "C:\Users\oliwi\computing\ollama-key.pem" \
ec2-user@ec2-54-93-101-2.eu-central-1.compute.amazonaws.com
```

7.2 Task 1: Hardware Inspection

After connecting, we inspected the remote machine's hardware:

```
lscpu
free -h
ip a
```

Key findings compared to our local machine:

- **CPU:** Intel Xeon Platinum 8259CL @ 2.50GHz, 2 vCPU (vs AMD Ryzen 7 with 16 threads locally).
- **RAM:** Only 916 MiB total – much less than our local 15.28 GiB.
- **Swap:** 0 B – none by default.
- **Private IP:** 172.31.42.172
- **No GPU:** t3.micro has no GPU, CPU-only inference.

7.3 Task 2: Installing Docker and Docker Compose

We installed Docker on the EC2 instance:

```
sudo dnf update -y
sudo dnf install -y docker
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ec2-user
```

We then installed Docker Compose manually:

```
sudo curl -L \
  "https://github.com/docker/compose/releases/latest/download/\
  docker-compose-linux-x86_64" \
  -o /usr/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose
docker-compose --version
# Docker Compose version v5.1.4
```

Note on Docker image size

We attempted to run Ollama inside a Docker container on EC2, but the official `ollama/ollama:latest` image is 3.4 GB – too large for the 8 GiB disk after the OS and Docker overhead. We therefore installed Ollama directly on the instance using the official install script, which is the recommended approach for resource-constrained machines.

7.4 Task 3: Uploading Files and Running Containers

We transferred the `docker-compose.yml` from Phase 3 to the EC2 instance using `scp`:

```
scp -i "C:\Users\oliwi\computing\ollama-key.pem" \
  "C:\Users\oliwi\computing\phase3\docker-compose.yml" \
  ec2-user@ec2-54-93-101-2.eu-central-1.compute.amazonaws.com:~/
```

We then installed Ollama directly:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Output confirmed: The Ollama API is now available at 127.0.0.1:11434 and WARNING: No NVIDIA/AMD GPU detected. Ollama will run in CPU-only mode.

7.5 Task 5: Pulling a Small Model

Due to the severe RAM constraint (916 MiB), we pulled the smallest available model as recommended by the project handout:

```
ollama pull smollm:135m
ollama list
```

NAME	ID	SIZE	MODIFIED
smollm:135m	b0b2a4617438	91 MB	13 seconds ago

At only 91 MB, smollm:135m fits comfortably within the available RAM. We verified the API was working:

```
curl http://localhost:11434/api/version
# {"version":"0.30.6"}
```

7.6 Task 6: Adding Swap Space

With only 916 MiB of RAM, running any model risks out-of-memory errors. We added 1 GiB of swap as a safety net:

```
sudo fallocate -l 1G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
free -h
```

Answers to the guiding questions

- **How does the EC2 hardware compare to your local machine?** The EC2 t3.micro has 2 vCPU and only 916 MiB RAM, compared to 16 threads and 15.28 GiB locally. The cloud machine is much weaker for AI inference.
- **Why use such a small model?** Because smollm:135m at 91 MB is the only model that fits comfortably in 916 MiB of RAM. Larger models would cause out-of-memory errors.
- **What is swap and why add it?** Swap is disk space used as overflow RAM. Without it, the OS would kill processes that exceed available memory. With 1 GiB of swap, the system can handle brief memory spikes without crashing.
- **Why could the Docker image not fit?** The official Ollama Docker image is 3.4 GB uncompressed. With only 8 GiB of disk and 2 GiB used by the OS, there was not enough space to unpack the image layers.
- **What is the private IP and why does it differ from the public DNS?** The private IP (172.31.42.172) is the address within AWS's internal network. The public DNS (ec2-54-93-101-2.eu-central-1.compute.amazonaws.com) is how external machines reach the instance. They are different because AWS uses NAT to map public addresses to private ones.

7.7 Optional Token 6: Latency Benchmark – Local vs Cloud

To compare inference performance between our local machine and the EC2 cloud instance, we wrote a Python script that sends the same prompt to both endpoints and measures response time and throughput.

```

import requests, time

LOCAL_URL = "http://localhost:11434/v1/chat/completions"
CLOUD_URL = "http://ec2-54-93-101-2.eu-central-1.compute.\
amazonaws.com:11434/v1/chat/completions"
PROMPT = "What is a CPU? Answer in one sentence."

def measure(url, model, label):
    payload = {"model": model,
               "messages": [{"role": "user", "content": PROMPT}],
               "stream": False}
    start = time.time()
    r = requests.post(url, json=payload)
    elapsed = time.time() - start
    data = r.json()
    tokens = data["usage"]["completion_tokens"]
    print(f"{label}: {elapsed:.2f}s | {tokens} tokens | "
          f"{tokens/elapsed:.2f} t/s")

measure(LOCAL_URL, "llama3.2:1b", "LOCAL_11434(llama3.2:1b)")
measure(CLOUD_URL, "smollm:135m", "CLOUD_EC2(smollm:135m)")

```

Results

Setup	Time (s)	Tokens	Tokens/s
Local – Llama 3.2 1B	3.55	40	11.28
Cloud EC2 – SmolLM 135M	12.94	647	50.01

Table 2: Benchmark results: local vs cloud inference.

Discussion

The results reveal an interesting paradox: the EC2 instance achieved a higher raw throughput (50 tokens/s vs 11 tokens/s locally), but the total response time was much longer (12.94 s vs 3.55 s) because `smollm:135m` generated 647 tokens instead of 40. The local `llama3.2:1b` model gave a concise, one-sentence answer as instructed, while the tiny cloud model ignored the length constraint and produced a verbose, repetitive response.

This highlights two important points. First, raw tokens/s is not the only metric that matters – answer quality and instruction-following ability are equally important. Second, the network latency to Frankfurt (EU) added only a small overhead; the bottleneck was the model’s tendency to generate many tokens, not the network itself. A fair comparison would require the same model on both machines, which was not possible here due to the EC2’s 916 MiB RAM constraint.

8 Problems Encountered and Solutions

Throughout the project we encountered several technical problems. The following table summarises the most significant ones.

Problem	Cause	Solution
<code>ollama serve</code> failed with “port already in use”	On Windows, Ollama starts automatically as a background service after installation	No action needed; confirmed server was reachable via <code>curl /api/version</code>
<code>curl</code> quoting errors in PowerShell	PowerShell mangles backslashes and quotes in the handout’s Linux-style commands	Wrote JSON body to a file and used <code>-d "@file.json"</code> with <code>curl.exe</code>
Ollama log files empty on Windows	Ollama running as a service logs differently than expected	Ran <code>ollama serve</code> manually in foreground to capture live logs
No “Perfect fit” models in <code>llmfit</code>	Integrated GPU (AMD Radeon 780M) has only 0.5 GB VRAM and uses Vulkan backend, which <code>llmfit</code> marks as incompatible	Used best “Good” fit models (1–1.5B) running in CPU+GPU mode
Continue.dev not seeing <code>test.py</code> context	Agent mode was active instead of Chat mode	Switched to Chat mode and pasted code directly in the prompt
MCP server “Connection closed” error	Continue could not find <code>node</code> executable by name on Windows	Specified full path <code>C:\\Program Files\\nodejs\\node.exe</code> in config
Docker image too large for EC2	<code>ollama/ollama:latest</code> is 3.4 GB; EC2 only has 8 GiB disk	Installed Ollama directly via install script instead of Docker
EC2 Ollama not reachable from outside	Ollama binds to <code>127.0.0.1</code> by default	Set <code>OLLAMA_HOST=0.0.0.0</code> via <code>systemctl edit</code> and opened port 11434 in the security group
SSH timeout to EC2	Dynamic home IP changed between sessions	Updated the SSH inbound rule in the security group to the new IP

Table 3: Problems encountered and solutions applied.

9 Conclusions

This project gave us a concrete understanding of the infrastructure that underlies modern AI assistants. We started with a simple question – what happens when you send a prompt to a model? – and ended with a working system that we had built and debugged ourselves, from the local terminal all the way to a cloud virtual machine.

The most important lesson from Phase 1 is that hardware is the first constraint. Before downloading any model, `llmfit` showed us that our integrated GPU was effectively unusable for inference, and the Ollama logs later confirmed this: the AMD driver was too old, the GPU was dropped, and everything ran on CPU. This is not a failure – it is exactly the kind of decision that infrastructure engineers make in production, where you match the model size to the available hardware rather than the other way around.

Phase 2 showed us that the same HTTP API we called with `curl` is what powers a full coding assistant. Continue.dev is not magic; it is a client that formats prompts, sends them to Ollama, and displays the response. The logs made this visible: every chat message, every edit, every autocomplete suggestion produced a `POST /api/chat` entry. Understanding the client–server boundary made the whole system feel less like a black box.

Phase 3 demonstrated why containers matter. Packaging Ollama into Docker meant that the setup could be described in a single `docker-compose.yml` file and reproduced on any machine. The container also revealed a new hardware constraint: Docker Desktop limits memory to roughly half the host RAM, which affected which models we could load.

Phase 4 brought everything together on a real cloud machine. The EC2 instance had only 916 MiB of RAM and no GPU – far weaker than our laptop – which forced us to use the smallest available model (`smollm:135m`, 91 MB) and add swap space to avoid out-of-memory crashes. The benchmark in the optional task produced a counterintuitive result: the cloud instance achieved higher raw throughput (50 tokens/s) than our local machine (11 tokens/s), but took longer overall because the tiny model generated far more tokens than needed. This showed us that tokens/s alone is not a useful metric – instruction-following ability matters just as much.

Across all four phases, the recurring theme was the same: AI infrastructure is a stack of ordinary components (HTTP servers, containers, virtual machines, security groups) and the model is just one piece of it. Knowing how to configure, connect, and debug these components is what separates using an AI tool from understanding one.

10 AI Models Used

During this project we used the following AI models and tools to assist with report writing, debugging, and understanding concepts:

- **Claude (Anthropic)** – used to help structure the report, code, explain error messages, and generate LaTeX code.
- **The local models themselves** – Llama 3.2 1B, DeepSeek-R1 1.5B, Gemma 3 1B, and SmolLM 135M were used directly as part of the project tasks.

11 Bibliography

References

- [1] Ollama. *Ollama – Get up and running with large language models*. <https://ollama.com>. Accessed June 2026.
- [2] Continue.dev. *Continue – Open-source AI code assistant*. <https://continue.dev>. Accessed June 2026.
- [3] Jones, A. *llmfit – Hundreds of models & providers. One command to find what runs on your hardware*. <https://github.com/AlexsJones/llmfit>. Accessed June 2026.
- [4] Anthropic. *Model Context Protocol*. <https://modelcontextprotocol.io>. Accessed June 2026.
- [5] Amazon Web Services. *Amazon EC2 – Secure and resizable compute capacity*. <https://aws.amazon.com/ec2>. Accessed June 2026.
- [6] Docker Inc. *Docker – Accelerated container application development*. <https://www.docker.com>. Accessed June 2026.
- [7] Meta AI. *Llama 3.2 – Lightweight, text-only models*. <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices>. Accessed June 2026.

- [8] DeepSeek. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. <https://arxiv.org/abs/2501.12948>. Accessed June 2026.